

Response to the Reports and Decision Letter for OPRE-2025-04-1885

“Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints”

We thank the Area Editor, the Associate Editor, and the referees for the thoughtful, technically detailed, and constructive reports. The feedback was especially valuable in identifying the issues that mattered most for an *Operations Research* revision: the open-system objective, the GPU KV-cache memory mechanism, the role of endogenous memory growth, the proof logic for the threshold policies, and the interpretation of latency evidence. We have therefore revised the manuscript substantially, not only by correcting local notation and exposition issues, but also by reorganizing the model, theory, and experiments around these points. Before responding point by point, we first describe the major changes in the revised manuscript.

- **Reframed the objective with exogenous arrivals.** We revised Sections 2–4 to state the objective under exogenous arrivals more carefully. With fixed external arrivals, a policy in the stable region can handle the offered load. The operational question is whether scheduling can sustain larger arrival rates by controlling endogenous memory growth, avoiding eviction-induced restarts, and keeping delay bounded. This framing also separates two regimes: in underloaded regimes, especially near the load boundary, the goal is to approximate the fluid upper bound; in overloaded regimes, the experiments examine how effectively each scheduler exploits available service capacity rather than losing work to eviction and restart. Accordingly, the paper now uses token-level *effective throughput* for completed decode-token service net of evictions, reports request-level effective completion rate in the experiments, uses the fluid model to identify the stability region and the memory required to sustain it, and plots latency versus arrival rate in Section 6.
- **Rewrote the model and notation.** Section 2 now introduces prefill, decode, batching, KV-cache growth, the memory constraint, and the policy space narratively. The main notation is collected in the Notation Summary appendix, and the stage notation is separated from prompt lengths and output lengths. We also clarified that the memory constraint applies to the KV caches of all in-system requests retained on the GPU, not merely to requests in the current batch.
- **Clarified the time model and its scope.** We expanded the discussion around Equation (1), explaining how the linear memory-dependent iteration-time model fits the multi-stage serving model with endogenous memory growth. We added real-GPU validation in the Additional Experiments appendix, including direct A100 timing measurements and end-to-end GPU runs implemented with SGLang 0.5.7, an open-source inference infrastructure package (SGLang Team, 2024). The main policy comparisons remain Vidur simulations configured for A100 hardware; the real-GPU experiments provide direct timing calibration for the simulator and implementation evidence for the WAIT and Nested WAIT scheduling logic. We also explain why the model is well matched to prefill-decode disaggregated systems such as Splitwise and DistServe (Patel et al., 2024; Zhong et al., 2024), where separating prefill from decode makes aggregate KV-cache usage a natural state variable for iteration time. For richer calibrated service-time curves, the formal guarantees would require corresponding model-specific analysis, but the same scheduling perspective remains useful: admission and batching decisions must control the batch composition across prefill and decode stages because this composition drives both memory pressure and future iteration times.

- **Clarified the theoretical contribution.** We removed the misleading “heavy traffic” terminology from the narrative and now describe the guarantees as asymptotic guarantees. The main text now explains why the problem differs from classical formulations with exogenous service requirements: KV-cache memory grows endogenously with decode progress, evictions can restart partially completed work, and unknown output lengths are revealed only through the service trajectory. These features make the primitive state-action space substantially larger, because the scheduler must control both current batch composition and future memory growth. Without careful scheduling, evictions can occur frequently and waste effective throughput through repeated restart work. The WAIT and Nested WAIT thresholds reduce this high-dimensional state-action space to threshold and boundary queues that capture the memory-coupled dynamics.
- **Expanded and reorganized the experiments.** Section 6 now reports latency and effective-completion-rate curves over arrival-rate grids that include underloaded, near-overloaded, and overloaded regimes. The Additional Experiments appendix adds simulator-to-GPU validation on an A100, supplemental end-to-end real-GPU experiments, repeated-run variability near the transition from near-overloaded to overloaded, threshold-without-waiting comparisons, and a prefill-decode-disaggregated deployment study. For the real-data experiment, we compare the latency of different policies and use the effective-completion-rate curve to show how much offered load each policy can sustain. The real-GPU appendix further reports end-to-end scheduling performance on an A100 GPU using SGLang.
- **Updated positioning and scope.** The Introduction and related-work discussion now clarify that our contribution is a modeling and scheduling contribution for multi-stage LLM inference with endogenous memory growth, not a claim of a new heavy-traffic limit. We also added discussion of concurrent theoretical work on LLM inference scheduling and positioned our model as complementary: our focus is on memory-dependent iteration times, GPU KV-cache growth, and eviction-induced restarts under the partial information available to the scheduler.
- **Improved exposition and organization.** The comments on exposition led to a substantial rewrite of Sections 1–6 and the appendices. We removed ambiguous terminology, introduced notation more gradually, added more detailed explanations and instances to show the importance of the problem and where the main interests and difficulties arise, and reorganized the appendix proofs to improve clarity.

Main concern	How the revised manuscript addresses it
Objective with exogenous arrivals	Clarifies the objective under exogenous arrivals with endogenous memory growth and a multi-stage decode mechanism: sustaining the offered load when feasible, avoiding eviction-induced restarts, and evaluating effective throughput, latency, and TTFT.
GPU KV-cache memory mechanism	Defines memory as the KV-cache usage of all admitted in-system requests retained on the GPU, distinguishes preemption from eviction, and makes restart costs explicit.
Linear time model	Explains the linear iteration-time model through aggregate KV-cache usage, reports A100 validation results, and discusses how the same state variable and scheduling insight can extend to prefill-decode-disaggregated deployments and related serving settings.
Theoretical contribution	Positions WAIT/Nested WAIT as fluid-guided threshold policies that control endogenous memory growth across prefill and decode stages, using threshold and boundary queues to reduce the high-dimensional state-action space under partial information.
Numerical experiments	Reports latency and effective-completion-rate curves over arrival-rate grids, adds long-decode and real-data experiments, repeated-run diagnostics near the transition from near-overloaded to overloaded, and supplemental real-GPU validation.
Writing and organization	Rewrites Sections 1–6, adds notation summary and examples, and reorganizes the appendices to improve proof clarity.

Response to the Area Editor

Comment. *The paper requires a complete expositional overhaul; the original presentation had notation inconsistencies, insufficient intuition, and proof summaries that were too high level to be useful.*

Our response. The Area Editor’s guidance was very helpful in making clear that the revision had to improve the paper’s exposition at the level of structure, not only wording. The main difficulty in the original presentation was that it introduced the mathematical abstraction before the reader had enough operational context for the serving mechanism. We have therefore reorganized the exposition so that the reader first sees the inference stages, batching decisions, KV-cache memory mechanism, and eviction/restart behavior, and then sees how these primitives lead to the model, policies, and analysis. Section 2 now introduces notation only as it enters the model, with a consolidated notation table moved to the Notation Summary appendix. Sections 4 and 5 now include step-by-step instances and mechanism-level explanations before the WAIT and Nested WAIT algorithms, so that the batching logic, memory-growth effect, and role of waiting are easier to see directly. The theorem discussions explain the memory conditions through batch composition, KV-cache growth, and threshold and boundary queues. We also reorganized the appendices so that the main proof steps are easier to follow and the text explains the verification path before deferring details.

Changes in the revised manuscript.

- Rewrote Section 2 around prompts, inference stages, batching, memory, and the objective.
- Added a notation summary in the Notation Summary appendix.
- Added examples and explanatory figures for WAIT and Nested WAIT in Sections 4 and 5.
- Reorganized the proof appendix into modular sections for Propositions and Theorems 1–3.
- Added theorem-interpretation paragraphs that connect each guarantee to the primitive stochastic and memory mechanisms.

Comment. *The model assumptions need either modification or better justification, especially the linear iteration-time model and the treatment of memory, OOM, and preemption.*

Our response. The model-assumption comments were particularly important because the memory mechanism is the paper’s main analytical difficulty. In the revision, we retained a tractable linear model but made its scope explicit. The iteration-time model separates the fixed per-iteration overhead from the marginal cost induced by aggregate KV-cache usage. Heterogeneous-prefill regimes may require richer calibrated service-time curves, whereas prefill-decode disaggregated deployments make aggregate decode-side KV-cache usage a natural state variable (Patel et al., 2024; Zhong et al., 2024). The revised text also explains why the scheduling insight extends beyond the linear specification: even under a richer service-time curve, the scheduler still has to regulate batch composition across prefill and decode stages, because that composition determines memory pressure and future batch service times. To avoid relying only on simulation, the Additional Experiments appendix compares Vidur predictions (Agrawal et al., 2024) with direct A100 measurements (NVIDIA, 2026) on a single-type calibration grid, and separately reports end-to-end experiments on an A100 GPU using SGLang (SGLang Team, 2024). The memory model has also been clarified: admitted requests that remain active keep their KV caches on the GPU and count against the memory constraint. If memory is exceeded, the server evicts and restarts an in-progress request, which is counted as lost useful work in the effective-throughput metric.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the linear iteration-time model and its KV-cache-driven scope.
- Revised Section 2.3 to state that the memory constraint applies to all in-system KV caches retained on the GPU.
- Added real-GPU validation in the Additional Experiments appendix, with A100 measurements and a fitted iteration-time model.
- Added a prefill-decode-disaggregated experiment in the Additional Experiments appendix showing that the same threshold logic applies in a decode-only serving setting.
- Revised Section 6 and the long-decode eviction table to report eviction-induced restart rates where relevant.

Comment. *The theoretical novelty should be clarified. If the contribution is modeling and problem structuring rather than a new stochastic-process theorem, that should be stated clearly.*

Our response. This guidance was very helpful in sharpening how we present the contribution. The revised manuscript no longer presents the contribution as a heavy-traffic theorem. Instead, it emphasizes the modeling and algorithm-design contribution: LLM inference creates a high-dimensional control problem because memory consumption grows rapidly with decode progress, output lengths may be unknown, and eviction can erase partially completed work. WAIT and Nested WAIT impose threshold structure that keeps the system close to the fluid equilibrium and reduces the analysis to threshold and boundary queues. The analysis then formalizes how this structure controls memory growth under the stated memory conditions and lets the stochastic policy approximate the fluid benchmark asymptotically.

Changes in the revised manuscript.

- Reframed Section 3 as a fluid model and equilibrium analysis rather than a throughput-maximization problem at a fixed arrival rate.
- Rewrote Sections 4.1–4.2 to explain why threshold-based admission reduces the state-action space.
- Rewrote Section 5 to explain how Nested WAIT handles unknown output lengths through segment boundaries and binomial thinning.
- Removed misleading “heavy traffic” language from the main narrative and replaced it with asymptotic language.

Comment. *The paper should better connect the LLM-inference domain to existing stochastic-modeling and scheduling literature.*

Our response. We appreciate the editor’s suggestion to connect the LLM-inference domain more directly to existing stochastic-modeling and scheduling literature. Following this guidance, the revised Introduction now discusses concurrent theoretical work on online scheduling, competitive ratios, regret, robust prediction, and stochastic batch processing for LLM inference. We also sharpened the distinction between our setting and classical formulations with exogenous service requirements. In our model, admitted prompts move through prefill and decode stages, their KV-cache requirements grow rapidly with decode progress, and the system may restart work due to endogenous memory growth. The scheduler therefore controls not only which jobs enter service, but also the batch composition across prefill and decode stages. This control problem created by endogenous memory growth motivates the fluid-guided threshold structure developed in the paper. We also discuss prefill-decode disaggregated systems (Patel et al., 2024; Zhong et al., 2024), where aggregate decode-side KV-cache usage is a natural state variable for scheduling.

Changes in the revised manuscript.

- Expanded the Other Related Work subsection in the Introduction.
- Added discussion of concurrent LLM-inference scheduling theory and the distinction between their timing/information models and our memory-dependent batch model.
- Added connections to Splitwise and DistServe-style prefill-decode disaggregation.

Response to Referee 1

We thank Referee 1 for the careful reading and for highlighting both the promise of the modeling direction and the aspects of the formulation that required clearer exposition. The comments on modeling clarity, notation consistency, and memory-overflow safeguards directly shaped the rewrite of Section 2 and the GPU KV-cache memory discussion throughout the paper.

R1.1. Memory overflow and KV-cache capacity

Comment. *The referee asks how the proposed algorithms ensure that KV-cache usage does not exceed capacity, especially under heavy workloads and uncertain output lengths.*

Our response. This was a central modeling point for the revision. The original manuscript did not state the GPU KV-cache memory mechanism explicitly enough. In the revised manuscript, memory is the total KV cache retained by all admitted in-system prompts on the GPU, not only the requests selected in the current batch. If the system admits too much work, decode progress increases KV-cache usage and may force the server to evict and restart an in-progress request. The revised model states the eviction order used in the simulations: under the last-in-first-out rule, the most recently admitted GPU-retained prompts are evicted first, and each evicted prompt restarts from prefill. The proposed policies control this risk by regulating how many prompts enter and advance through decode. For known output lengths, WAIT uses type-specific thresholds derived from the fluid equilibrium and avoids eviction under the stated memory condition. For unknown output lengths, Nested WAIT adds segment-level thresholds and a moderate safety buffer so that prompts that survive to later decode segments remain within the memory budget with high probability.

Changes in the revised manuscript.

- Added the memory-growth example and GPU KV-cache memory discussion in Section 2.3.
- Added Example 2 in Section 4 to show how FCFS can trigger eviction cascades when KV-cache growth pushes the batch composition away from the fluid equilibrium, even when the nominal capacity is sufficient.
- Added Example 3 and the Nested WAIT explanation in Section 5 to show how unknown output lengths are revealed through segment boundaries and how segment-level thresholds control downstream memory growth.
- Revised Algorithms 1 and 2 and their explanations to state how threshold admission, segment boundaries, and retained KV caches interact with the memory constraint.
- Added a long-decode near-overloaded eviction diagnostic in Section 6, where Sarathi incurs a positive eviction-induced restart rate while WAIT has no eviction-induced restarts in the reported run.

R1.2. Linearity of Equation (1)

Comment. *The referee asks for additional justification of the linear iteration-time assumption and for discussion of possible nonlinear relationships across GPU architectures.*

Our response. The referee’s comment helped us clarify both the scope of Equation (1) and the reason it is useful for the scheduling problem. The linear form separates a fixed per-iteration overhead from the marginal cost induced by aggregate KV-cache usage. The revised manuscript now states that this is a calibrated approximation for the regimes studied in the paper, not a universal timing law across all hardware and workload mixes. Mixed prefill-decode iterations, especially with long or highly heterogeneous prefill lengths, may require additional prefill-attention or hardware-specific terms. We added two validation checks: Section 2 reports direct A100 profiling of batch inference time against total cached tokens (NVIDIA, 2026), and the Additional Experiments appendix compares Vidur’s iteration-time predictions (Agrawal et al., 2024) with direct A100 measurements. The revision also explains that the scheduling insight is not tied to treating the linear specification as universal: under richer calibrated service-time models, batch composition across prefill and decode stages remains the key control variable linking admission decisions to current memory usage, future memory growth, iteration times, and eviction risk.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the fixed-overhead and marginal KV-cache components of Equation (1).
- Added A100 batch-inference-time profiling in Section 2 and Additional Experiments appendix validation comparing Vidur predictions with direct A100 measurements. The appendix distinguishes the Llama-2-7B profiling-supported batch-size range $B \leq 128$ and reports a separate test of extrapolation to batch size $B = 256$.
- Clarified where the linear timing model is intended to apply, and noted that mixed prefill-decode iterations and heterogeneous prefill lengths may require additional calibrated timing terms.
- Added discussion that richer calibrated service-time curves can be incorporated conceptually, while the formal results in this paper focus on the linear multi-stage model with endogenous memory growth.

R1.3. Notation inconsistencies

Comment. *The referee notes that the original manuscript used similar notation for decode stage, prompt length, output length, and current processing stage.*

Our response. This comment identified a genuine source of confusion in the original manuscript. The revised manuscript separates the relevant concepts: the prefill/decode stage is denoted by the stage index s , while prompt length, output length, and type-dependent lengths use separate notation. To reduce the burden on the reader, we moved the full notation table to the Notation Summary appendix and introduce symbols narratively as they first appear in the main text.

Changes in the revised manuscript.

- Revised Section 2 to distinguish prefill/decode stage notation from prompt length and output length notation.
- Added the Notation Summary appendix table collecting the main symbols used throughout the paper.
- Updated the algorithms and theorem statements to use the same stage and threshold notation.

R1.4. Policy space and batching operations

Comment. *The referee asks for a clearer mathematical characterization of the policy space Π and the batching operations implied by a policy.*

Our response. This was a useful point because the original manuscript introduced π before the admissible decisions were fully specified. We revised this part so that the policy class is explicit before the algorithms are introduced. The revised manuscript now includes a dedicated objective and policy-space subsection. It states what the policy observes at a decision epoch, how it forms batches, how preemption and eviction are treated, and what information is unavailable when output lengths are unknown. This makes WAIT and Nested WAIT concrete elements of the admissible policy class, with their batching and memory decisions specified within the model.

Changes in the revised manuscript.

- Revised Section 2.4 to define the objective, the policy class, the information available to the scheduler, and the main performance metrics.
- Clarified the relation between policy decisions, batch formation, preemption, eviction, memory consumption, latency, TTFT, and effective throughput.

R1.5. Need for intuition after definitions and algorithms

Comment. *The referee asks for more explanatory text after key definitions and Algorithm 1.*

Our response. This suggestion was very helpful for making the algorithms easier to read before the formal analysis. Section 2 now explains each modeling component through the running LLM-inference example. Section 4 begins with a concrete FCFS failure example before introducing WAIT, showing why threshold admission is needed to control batch composition and KV-cache growth. Section 5 begins with an unknown-output-length example before introducing Nested WAIT, showing how segment boundaries reveal output-length information gradually. The algorithm descriptions now explain why thresholds regulate both admission and downstream batch composition across prefill and decode stages.

Changes in the revised manuscript.

- Added explanatory text around the introductory inference figure and the batching example figure.
- Added Example 2 before WAIT to show how FCFS can lose balance in batch composition and trigger eviction-induced restarts.

- Added Example 3 before Nested WAIT to show how unknown output lengths are handled through segment-boundary revelation.
- Added mechanism-level paragraphs after Algorithm 1 and Algorithm 2 explaining how the thresholds regulate admission and downstream decode-stage advancement.

R1.6. Typos

Comment. *The referee notes several typos, including d versus d_1 and “Throughput” versus “Throughput.”*

Our response. We corrected these issues and appreciate the referee pointing them out. Because the notation issues affected more than isolated typos, we also conducted a broader consistency check for notation, terminology, cross-references, and repetitive phrasing.

Changes in the revised manuscript.

- Corrected “Throughput”/“Throughput” and related throughput notation.
- Corrected d , d_0 , and d_1 inconsistencies in the model and fluid sections.
- Removed duplicate or inconsistent notation flagged in the referee comments.

Response to Referee 2

We sincerely thank Referee 2 for the detailed, constructive, and encouraging report. The report was especially helpful because it combined a supportive evaluation of the paper’s modeling contribution with precise technical suggestions on the time model, the memory assumption, the proof structure, and experimental reproducibility. These suggestions led to several substantive improvements in both the main text and the appendix.

R2.1. Scope of the linear iteration-time model

Comment. *The referee suggests that Equation (1) may omit prefill attention, linear layers, and decode attention terms, and asks which asymptotic regime the paper targets.*

Our response. This comment was very helpful in sharpening the scope of Equation (1). The revised manuscript now states that the linear iteration-time model is intended for regimes in which marginal iteration time is primarily driven by aggregate KV-cache usage. We also explain what the model abstracts from: mixed prefill-decode iterations, long or highly heterogeneous prefill lengths, and hardware-specific kernels may require additional prefill-attention, linear-layer, or calibrated service-time terms. Rather than claiming universality, we position Equation (1) as a transparent model of the admission-control trade-off central to the paper: admitting more prompts improves batching opportunities, but also increases iteration time and

memory pressure through larger KV caches. For richer calibrated service-time curves, the same fluid-guided construction would start from the corresponding fluid equilibrium and memory-feasibility condition. Batch composition across prefill and decode stages would still be the key control variable linking admission decisions to current memory usage, future memory growth, iteration times, and eviction risk. We also added A100 validation (NVIDIA, 2026).

Changes in the revised manuscript.

- Revised Section 2.2 to state the operating regime of Equation (1), the timing terms it abstracts from, and how richer calibrated service-time curves would change the fluid equilibrium and memory-feasibility calculation.
- Replaced the earlier L20 profiling plot with an A100 profiling figure for Llama-2-7B (NVIDIA, 2026; Hugging Face, 2025), and added an appendix validation comparing Vidur predictions with direct A100 measurements, including a separate extrapolation check to $B = 256$.
- Added a prefill-decode-disaggregated experiment in the Additional Experiments appendix to illustrate how the same threshold insight extends to settings where decode-side KV-cache usage is isolated.

R2.2. Clarifications in Section 2

Comment. *The referee provides several minor comments on notation, prefill throughput, the policy class, and whether preemption is permitted.*

Our response. These detailed comments pointed to several places where Section 2 imposed unnecessary burden on the reader. In the revision, the stage notation is separated from prompt and output lengths, the roles of prefill and decode are described explicitly, and the objective discussion now clarifies the metrics studied in the paper. The revised model does not rely on costless swapping of inactive KV caches. Requests that are admitted and remain active retain their KV caches on the GPU and count toward memory consumption; if memory is exceeded, the server evicts and restarts in-progress work, which is treated as lost useful service. The revised text also states the last-in-first-out eviction rule used in the simulations: the most recently admitted GPU-retained prompts are evicted first, and each evicted prompt restarts from prefill.

Changes in the revised manuscript.

- Rewrote Section 2.1–2.4 and added the Notation Summary appendix to separate prefill/decode stage notation from prompt and output lengths.
- Clarified the admissible policy decisions, including admission, batch formation, preemption, eviction, and the partial information available to the scheduler.
- Clarified that the memory constraint applies to retained KV caches of admitted in-system requests, including requests not selected in the current batch, and stated the last-in-first-out eviction and restart-from-prefill convention used when memory is exceeded.

R2.3. The assumption $C \geq M^*$

Comment. *The referee notes that $C \geq M^*$ may be a nontrivial requirement, especially for very long outputs, and asks whether the experiments satisfy or violate this condition.*

Our response. The point about $C \geq M^*$ is important, especially for long outputs. We now treat M^* as the memory requirement for the fluid equilibrium, not as a benign capacity assumption. The condition $M^* \leq C$ says that the workload is stabilizable in the fluid model. The policy question is whether an online scheduler can realize this stability in the stochastic system while memory grows rapidly with decode progress. To connect this condition to observed system behavior, the revised experiments examine underloaded, near-overloaded, and overloaded regimes across synthetic and heterogeneous workloads. We also added a real-data memory check: for the real dataset studied in Section 6, the manuscript reports representative values of $M^*(\lambda)$ computed from the empirical prefill/decode length distribution and compares them with the memory capacity C . Across these regimes, the empirical patterns are consistent with the theory’s mechanism: threshold-based admission helps the system exploit available memory and server capacity while reducing eviction-induced restarts. As memory pressure increases, WAIT and Nested WAIT keep completed work closer to usable capacity by regulating when prompts enter and advance through decode, rather than relying on excess memory.

Changes in the revised manuscript.

- Revised the discussion around M^* in Sections 3 and 4 to present it as the fluid-equilibrium memory requirement rather than a benign capacity assumption.
- Added a real-data $M^*(\lambda)$ versus C table in Section 6 for the real dataset.
- Added long-decode experiments in Section 6, including p512d1000, to test near-overloaded and overloaded behavior under severe memory pressure.

R2.3a. Segment-wise design for unknown or long output lengths

Comment. *The referee notes that maintaining a separate stage for every possible output length may be impractical in long-output settings.*

Our response. This comment helped us separate the analytical construction from the implementation-oriented segment design. The main theorem presents the transparent case in which output lengths map to segment boundaries. The Extensions appendix then discusses general segment designs that group nearby output lengths into fewer segments. This is the form used in the real-data experiments: segment-level thresholds preserve the same continuation-probability logic while avoiding the need to maintain a separate threshold for every individual decode length.

Changes in the revised manuscript.

- Revised Section 5 to explain segment boundaries as the points where output-length information is revealed.

- Added Extensions appendix discussion of general segment design and coarser segmentations.
- Revised Section 6.2 to describe how empirical continuation probabilities determine the segment thresholds in the real-data workload.

R2.4. Proof of Theorem 1 and decode-stage dynamics

Comment. *The referee asks how the proof accounts for l' decode stages and whether completions occur only after enough batch executions.*

Our response. The proof now gives a more explicit account of the decode-stage dynamics. WAIT is designed so that once a type is scheduled, threshold-sized batches advance through the decode stages under a controlled service-opportunity coupling. This threshold structure is what permits the proof to reduce the multi-stage process to a boundary queue for each type. The main text now explains this dimensionality reduction before the theorem discussion, and the appendix proof explicitly separates the batch index from the decode-stage index.

Changes in the revised manuscript.

- Revised Section 4.2 to explain how threshold-sized batches move through decode stages.
- Revised the proof appendix for Theorem 1 to define the batch index separately from the stage index.
- Added explanatory text connecting queue accumulation to throughput loss in the proof of Theorem 1.

R2.5. Thresholds without waiting

Comment. *The referee asks what happens if the algorithm uses thresholds but does not wait for enough prompts to accumulate.*

Our response. To isolate the role of waiting, we added a targeted comparison in the Additional Experiments appendix. The threshold-only variant imposes an upper bound on in-system requests but does not wait to form threshold-sized batches. The comparison shows that the waiting component has a regime-dependent role. At low arrival rates, waiting can add modest delay because the server has sufficient capacity to process requests immediately. In near-overloaded regimes, waiting becomes useful because it regulates when work enters service and improves batch formation. Once the system is overloaded, the aggregate upper bound dominates and the two variants behave more similarly.

Changes in the revised manuscript.

- Added the “Threshold without waiting” subsection in the Additional Experiments appendix.
- Added the wait-versus-threshold-only figure reporting the latency comparison across arrival rates.

R2.6. Role of the asymptotic assumption

Comment. *The referee asks what role the original heavy-traffic assumption plays and whether the theorem can be stated without it.*

Our response. We appreciate the referee making this distinction explicit. The analysis does not take a classical heavy-traffic limit in which the load approaches the boundary of the stability region. The revised manuscript now states the scaling directly: arrival rates are multiplied by ζ , processing times are divided by ζ , and the physical memory capacity C is kept fixed. This scaling averages out stochastic fluctuations while preserving the memory constraint, and the theorem reports throughput gaps and service-normalized delay metrics over the effective horizon ζT . We have removed the heavy-traffic framing from the narrative and now describe the result as an asymptotic guarantee relative to the fluid benchmark, not as a conventional diffusion limit.

Changes in the revised manuscript.

- Replaced “heavy traffic” terminology in the main text with asymptotic terminology.
- Revised the theorem discussion in Sections 4 and 5 to state the role of the limit more directly.

R2.7. Formula and proof-detail comments

Comment. *The referee lists several formula checks and appendix proof clarifications, including d_0 versus d_1 , M^π versus M^* , undefined notation, and an alternative proof idea.*

Our response. We treated these formula and proof comments as guidance for revising the appendix. The revised manuscript corrects the d_0/d_1 inconsistencies, clarifies the distinction between M^* and M^π , defines segment probabilities and safety-buffer terms before the Nested WAIT theorem, and reorganizes the proof appendix around the main theorem statements. We also followed the referee’s suggested proof direction by making the coupling argument more direct and by separating the auxiliary embedded process from the original event-driven system.

Changes in the revised manuscript.

- Corrected the d_0/d_1 inconsistencies and clarified the distinction between M^* , M^π , and the safety-buffer terms.
- Revised Section 5.2 to define p_k , n_k , M^π , and the safety-buffer interpretation before Theorem 2.
- Reorganized the proof appendix around theorem-specific arguments and added citations to established stochastic-process tools used in queueing analysis.

Detailed correction checklist. We also checked the detailed notation and proof points in the report. In particular, the revision (i) separates prompt type $j(i)$, prefill/decode stage s_i^t , prefill length l_j , and decode length l'_j ; (ii) states that $s_i^t = 0$ denotes prefill; (iii) corrects the d_0/d_1 inconsistencies in the fluid calculations; (iv) defines $\Delta T(\cdot)$, M^* , and M^π before they are used in the theorem statements; (v) states the advancement logic through prefill and decode used in Theorem 1; (vi) distinguishes the embedded review intervals used

in the WAIT proof from actual event-driven batches; (vii) adds the memory-state and post-review boundary-queue conventions in the WAIT and Nested WAIT proofs; (viii) cites standard reflected-random-walk and Kingman-type tools rather than reproving those facts from first principles; and (ix) removes or corrects the undefined or ambiguous symbols flagged in the appendix.

R2.7a. Appendix organization and use of standard stochastic results

Comment. *The referee and the Area Editor suggest that the appendix should be easier to verify and should connect more directly to standard stochastic-process tools.*

Our response. We reorganized the proof appendix so that its structure is easier to verify. It now separates proposition proofs, the WAIT theorem proof, the Nested WAIT theorem proof, the time-varying extension, and supporting lemmas. Within the theorem proofs, we use standard stochastic-process tools explicitly: a Lindley-type recursion for the threshold queue, random-walk maximum bounds for queue accumulation, and martingale bounds for the Nested WAIT safety buffer. The main text now explains why the policy creates the reduced process before the appendix analyzes it.

Changes in the revised manuscript.

- Split the proof appendix into theorem-specific proof sections.
- Added definitions of the batch index, stage index, and coupled process before using them in proofs.
- Added citations to standard stochastic-process tools where they replace or streamline first-principles derivations.

R2.8. Experimental parameters and reproducibility

Comment. *The referee asks for batch-size, memory, capacity, Sarathi configuration, and Nested WAIT parameter details in Section 6.*

Our response. We substantially rewrote Section 6 and the Additional Experiments appendix so that the main settings are explicit rather than implicit. The main text now states the hardware, simulator, baseline configurations, WAIT/Nested WAIT threshold construction, arrival-rate grids used in the experiments, and number of replications. The baselines are described as greedy FCFS-based policies; Sarathi differs by using chunked prefill (Agrawal et al., 2023), and the memory-reservation rule used by both baselines is stated explicitly. We present the comparisons relative to these documented baseline configurations. We also added a memory-cap calculation for the A100 80 GB / Llama-2-7B configuration (NVIDIA, 2026; Hugging Face, 2025): after the model weights and the baseline memory margin, the baseline-memory runs use a KV-cache cap of approximately 1.37×10^5 tokens. The token calculation follows the standard key/value-cache accounting for Transformer decoding (Atharva, 2026). The same paragraph now states that if retained GPU KV-cache usage exceeds this cap, the server evicts and restarts in-progress requests.

We also introduced an additional batch-cap control parameter tl to make the experimental threshold construction explicit. This parameter is the system-wide threshold scale used to construct the integer scheduling

thresholds, so the implementation does not manually tune a separate threshold for every prefill/decode stage or segment. In a single-type WAIT run, this scale is distributed across the prefill and decode stages of the workload and converted into the per-stage limits used by the scheduler. In Nested WAIT, the same scale is first allocated across decode-stage segments using segment lengths and empirical continuation probabilities; the resulting segment-level caps are then rounded and spread across the stages inside each segment. For example, in the two-type workload with decode segments 1–20 and 21–50, the second segment is reached by the long-decode 30% of requests. With $tl = 30$, the construction solves $20n_1 + 30n_2 = tl$ with $n_2/n_1 = 0.3$, giving segment-level caps of approximately 21 and 9 before integer rounding. Thus tl is not itself a memory capacity and does not mean that tl longest-context requests can reside on the GPU simultaneously. Actual retained KV-cache usage depends on realized prompt lengths and decode stages and remains governed by the memory cap. For the real-data experiment, the manuscript reports the $tl \in \{20, 40, \dots, 200\}$ parameter grid, segment-count grid $L \in \{1, 2, 3, 4, 5, 10, 20\}$, and margin parameter $\eta = 0.05$ used in this calibration. We present this as a rate-aware calibration of a distribution-aware policy: the workload distribution and offered arrival rate set the parameters before online scheduling, while each scheduling decision does not use the realized final output length of the request being scheduled. The GPU appendix separately reports rate-specific parameter settings for the real-data GPU runs.

Changes in the revised manuscript.

- Added hardware, memory-cap, baseline-configuration, and WAIT/Nested WAIT parameter paragraphs at the start of Section 6.
- Revised the real-data subsection to explain the dataset, sampling, segment construction, distribution-aware thresholds, and threshold allocation.
- Added real-GPU implementation details in the appendix, including the rate-specific parameters used in the real-data GPU run.

R2.9. Simulation fidelity and real-GPU validation

Comment. *The referee recommends validating Vidur against actual GPU measurements.*

Our response. We added the requested validation in two complementary forms. The Additional Experiments appendix compares Vidur predictions with direct A100 measurements for Llama-2-7B on a single-type calibration grid. The raw Vidur profiling data for this Llama-2-7B/A100 setting cover batch sizes through $B = 128$; we report the profiled range and separately test extrapolation to batch size $B = 256$ against a direct A100 measurement. The fitted validation curve has small mean absolute percentage error and high R^2 , and the extrapolation to $B = 256$ remains close to the measured A100 runtime. We also added end-to-end real-GPU experiments using SGLang 0.5.7 (SGLang Team, 2024), comparing WAIT with the baseline scheduler for a single-type workload and Nested WAIT for the real-data workload distribution. The main policy comparisons in Section 6 remain Vidur simulations configured for A100 hardware, while these real-GPU experiments provide direct timing calibration for the simulator and implementation evidence for the WAIT/Nested WAIT scheduling logic.

Changes in the revised manuscript.

- Added the simulator-calibration figure in the real-GPU validation appendix.
- Added end-to-end real-GPU comparisons on single-type and real-data workloads.
- Added references to the validation from Section 6.

R2.10. Long outputs

Comment. *The referee asks how the algorithm behaves when output length exceeds 1000 tokens.*

Our response. This is a natural stress test for the proposed memory-control mechanism, and we added a long-decode workload to Section 6. The workload p512d1000 keeps the prefill length fixed and increases the decode length to 1000. This setting stresses KV-cache growth and causes all policies to lose stability at much lower arrival rates. WAIT has the lowest latency near the boundary and avoids eviction-induced restarts in the near-overloaded diagnostic, illustrating that threshold-based admission remains useful when decode-side memory growth is pronounced.

Changes in the revised manuscript.

- Added the long-decode regime in Section 6.1.
- Added the long-decode figure and near-overloaded eviction table for latency, effective completion rate, and eviction-induced restart evidence.

Response to Referee 3

We are especially grateful for the detailed comments on the exogenous-arrival interpretation, latency evidence, and theoretical positioning. Because these concerns were also central to the paper’s exposition and revision, we address them in detail below. The comments identified several issues that were central to the revision: the throughput interpretation under exogenous arrivals, the use of heavy-traffic terminology, the need for meaningful latency experiments, and the need to explain why the theory is not merely a routine positive-recurrence argument. Addressing these points led to the largest changes in the revision.

D.1. Heavy-traffic terminology

Comment. *Referee 3 notes that the original “heavy traffic” analysis was not a conventional heavy-traffic limit; it was a long-horizon limit under fixed arrival rates.*

Our response. This comment helped us sharpen the asymptotic framing of the theory. The revised paper no longer presents the results as a classical heavy-traffic analysis in which the load approaches the boundary of the stability region. It now states the scaling directly: arrival rates are multiplied by ζ , processing times are divided by ζ , and the physical memory capacity C is kept fixed. This scaling averages out stochastic fluctuations while preserving the memory constraint, and the theorem reports throughput gaps and service-normalized delay metrics over the effective horizon ζT . We have removed the heavy-traffic framing from

the narrative and now describe the result as an asymptotic guarantee relative to the fluid benchmark, not as a conventional diffusion limit.

Changes in the revised manuscript.

- Replaced the “heavy traffic” framing with the asymptotic scaling used in the theorems.
- Revised the discussions after the WAIT and Nested WAIT theorems to state what the limit controls and how the delay metrics are normalized.

D.2. Throughput with exogenous arrivals

Comment. *Referee 3 points out that, with exogenous arrivals, a stable system completes work at the arrival rate, so the relevant question is the stability region rather than throughput at a fixed arrival rate.*

Our response. We revised the objective around the stable operating range and effective throughput under exogenous arrivals. The revised Section 2 objective discussion now explains that, within the stable region, a policy cannot complete more useful work than the offered load. The operational question is therefore which arrival rates a policy can sustain while avoiding idle capacity, memory overflow, and eviction-induced restarts. The nonclassical feature in this LLM inference system is endogenous KV-cache growth: a request’s memory usage grows as it advances through decode, so the scheduler must control the batch composition across prefill and decode stages, not only the external arrival rate. Section 3 uses the fluid model to identify the batch composition and memory requirement needed to support the fluid equilibrium. We also added three illustrative examples to make this distinction concrete. Example 1, “Dynamic Memory Growth and Eviction,” shows how retained KV caches grow during decode and why memory overflow creates eviction/restart costs. Example 2, “Eviction Cascade under FCFS,” shows how poor admission control can move the system away from the fluid equilibrium batch composition, trigger evictions, and waste partially completed decode work. Example 3, “Unknown Output Lengths,” shows how partial output-length information makes the memory-growth dynamics more difficult to control, because the scheduler must regulate continuation across decode segments before each request’s final output length is known. Section 6 reports mean latency and effective completion rate versus arrival rate, so the observed transition from near-overloaded to overloaded operation appears through latency growth and completion-rate saturation.

Changes in the revised manuscript.

- Revised Section 2.4 to define the admissible policy class, scheduler information, effective throughput, latency, TTFT, and the objective under exogenous arrivals.
- Revised Section 3 to connect the fluid stability region to the batch composition and memory requirement M^* needed to support the fluid equilibrium.
- Added the real-data $M^*(\lambda)$ versus C table in Section 6 to relate the empirical arrival-rate transition to the fluid stability region.
- Revised Section 6 to report mean latency and effective completion rate versus arrival rate, so the stable, near-overloaded, and overloaded regimes are visible in the same diagnostics.

The revision also clarifies the interpretation of M^* . It is the memory requirement needed to sustain the fluid equilibrium for the arrival vector under consideration, rather than a “throughput-optimal memory” level. For a fixed physical capacity C , the condition $M^* \leq C$ determines whether that load lies inside the fluid stability region. This distinction matters under exogenous arrivals: increasing memory does not raise completed work above the offered load in a stable system, but insufficient memory or poor admission control can shrink the stable operating range and reduce effective throughput through eviction-induced restarts.

D.3. Positive recurrence and theoretical significance

Comment. *Referee 3 argues that bounded latency under a stable policy may follow from standard positive-recurrence arguments unless the paper explains the nontrivial part of the analysis.*

Our response. This comment helped us sharpen the theoretical positioning. We have revised the theory sections to separate the familiar stability implication from the paper-specific control question. The difficult step is not that a stable queue has finite delays; it is designing a policy whose high-dimensional LLM-serving state can be controlled under endogenous KV-cache memory growth and eviction risk. The revised fluid section now makes the source of this difficulty explicit: for a single type, away from the critical denominator, the memory required to sustain the fluid equilibrium grows quadratically in the decode length, because both the number of active prefill/decode stages and the average KV-cache size across the batch composition increase. Thus even a small amount of long-decode traffic can dominate the memory condition defining the fluid stability region. The WAIT proof then connects the actual event-driven policy to an auxiliary embedded full-threshold process observed at deterministic review epochs. The sample-path coupling is the nonstandard step: it shows that controlling the threshold batch composition also controls the batch composition across prefill and decode stages, and that the embedded process tracks both entry starts and downstream stage advancement up to a negligible one-interval lag. Nested WAIT further handles unknown output lengths by using segment boundaries: prompts reveal their output length only when they complete at a boundary or survive to the next segment. The boundary-queue coupling exploits this structure to control memory overflow without tracking every prompt across every prefill/decode stage.

Changes in the revised manuscript.

- Added Section 3 discussion showing why long-decode requests can dominate the memory requirement for the fluid stability region.
- Rewrote Sections 4.2 and 5.2 to explain the threshold-queue and boundary-queue reductions created by WAIT and Nested WAIT.
- Reorganized the proof appendix around theorem-specific arguments, with explicit batch/stage indices, coupled processes, and citations to established stochastic-process tools used in queueing analysis.
- Revised the theorem interpretation paragraphs to connect the memory conditions to eviction prevention, dimensionality reduction, and service-normalized delay.

The revision therefore emphasizes that the policy is not merely a work-conserving batching rule. The threshold construction induces a lower-dimensional representation of the memory-coupled state. Without

this structure, the primitive state includes the number of prompts at each decode stage, their retained KV caches, output-length uncertainty, and the possibility of eviction-induced restarts. The threshold design is what creates the lower-dimensional threshold and boundary queues analyzed in the proof.

D.4. Latency experiments and overloaded regimes

Comment. *Referee 3 notes that latency comparisons in overloaded regimes can be misleading and recommends plotting mean latency versus arrival rate.*

Our response. We followed this recommendation by reworking the numerical section around arrival-rate curves. The main figures now plot mean end-to-end latency versus arrival rate and pair it with effective completion rate. Each latency point is computed over a central measurement window after the initial transient, using the same window across policies and replications. This presentation shows the underloaded, near-overloaded, and overloaded regimes in the same figure. In the stable region, where a policy can handle the offered load, latency is interpreted as the end-to-end delay comparison. Around the transition from near-overloaded to overloaded operation, latency growth indicates that the policy is operating close to its empirical capacity boundary over the simulated horizon. Above that range, effective completion rate and eviction-induced restarts show how much effective throughput each policy can still realize.

Changes in the revised manuscript.

- Rebuilt the main numerical comparisons as arrival-rate experiments for the single-type, two-type, long-decode, and real-data workloads.
- Reported latency together with effective completion rate, highlighting the near-overloaded regime where latency differences become most informative and separating it from overloaded completion-rate saturation and eviction-induced waste.
- Added repeated-replication summaries, a long-horizon check near the two-type transition, and long-decode eviction diagnostics under severe memory pressure.

The revised numerical section now separates low-load behavior from the near-overloaded and overloaded regimes. At low load, the policies are often close. In the near-overloaded and overloaded regimes, WAIT/Nested WAIT performs better because the threshold controls keep the batch composition closer to balance and reduce eviction-induced waste. For both the synthetic and real-data experiments, we report mean latency together with effective completion rate. The effective completion rate matches the offered arrival rate in the stable region and falls below it once the policy can no longer sustain the load.

D.5. Algorithmic contribution and latency

Comment. *Referee 3 characterizes WAIT as a pipeline-filling idea and asks for a stronger explanation of latency effects and algorithmic contribution.*

Our response. This comment was valuable because it identified a place where the original exposition could make WAIT look like a simple pipeline-filling rule. In the revision, we added problem-specific exposition and

examples to explain the additional control problem. In this setting, KV-cache memory grows endogenously with decode progress, admitted requests can later create overflow, and evictions restart partially completed work. These features make the state-action space high-dimensional: the scheduler must account for the number of requests across prefill/decode stages, their retained KV-cache memory, and the future overflow risk created by current admission decisions. WAIT uses threshold control to regulate this batch composition so that arrivals, completions, and memory growth remain balanced. This threshold structure then performs a dimensionality reduction: instead of tracking every request at every decode stage, the analysis can track whether each per-type prefill/decode-stage queue has accumulated enough requests to form a threshold-sized service batch. Nested WAIT extends the same principle when individual output lengths are unknown at admission: the scheduler does not require final-length information, and each request’s own decode trajectory reveals whether it continues to later segments. The resulting boundary queues track the residual requests carried across segment boundaries and play the same simplifying role for partial output-length information. The numerical section now reports latency versus arrival rate to show where these controls improve end-to-end performance.

Changes in the revised manuscript.

- Added and revised examples in Sections 4 and 5 to show how WAIT and Nested WAIT control concrete prefill/decode trajectories.
- Revised the algorithm discussions to emphasize endogenous memory growth, batch-composition control, and the resulting threshold/boundary-queue reductions.
- Added threshold-without-waiting and real-GPU comparisons in the Additional Experiments appendix as complementary evidence for the scheduling mechanism.

D.6. Memory constraint and preempted jobs

Comment. *Referee 3 asks whether the memory constraint applies only to requests in the current batch and whether paused requests continue to occupy GPU KV-cache memory.*

Our response. This comment highlighted an important modeling ambiguity. We revised the model to make the GPU KV-cache memory mechanism explicit. The memory constraint applies to the KV caches of all admitted in-system requests retained on the GPU, not only to requests selected in the current iteration. Requests that are not selected remain in their current queues, and those already in decode retain their KV caches. We do not rely on moving GPU-resident KV caches to CPU memory or SSD at zero cost. If the retained GPU KV-cache memory exceeds capacity, the serving system must evict and restart an in-progress request, which can lead to significant effective-throughput loss because partially completed decode work is discarded. The revised model specifies the last-in-first-out eviction rule used in the simulations: the most recently admitted GPU-retained prompts are evicted first, and each evicted prompt restarts from prefill. This loss channel motivates the admission-control problem studied in the paper.

Changes in the revised manuscript.

- Revised Section 2.3 to state that the memory constraint covers all GPU-retained KV caches of admitted

in-system requests.

- Revised the Algorithm 1 and Algorithm 2 explanations to state that unselected decode prompts retain their KV caches.
- Added the last-in-first-out eviction rule, the restart-from-prefill convention, and the corresponding eviction-induced restart diagnostics.

D.7. Writing quality, definitions, and interpretation

Comment. *Referee 3 notes that the original manuscript was unclear, with incomplete definitions, little interpretation of formulas, and notation inconsistencies.*

Our response. We are grateful for this comment, which became a central guide for the revision. It led us to treat the exposition as a structural issue rather than a matter of local editing. The revised manuscript now develops the fluid model step by step, with the main formulas followed by their operational interpretations. The algorithms are introduced through concrete examples before pseudocode, so the reader sees the scheduling mechanism before the formal implementation. The theorem statements are followed by paragraphs explaining what the memory terms mean and how the proof reduces the original system to threshold and boundary queues.

Changes in the revised manuscript.

- Rewrote Sections 2–5 to improve notation, flow, and interpretation of the model, fluid equilibrium, and algorithms.
- Added a notation table and explanatory paragraphs after the fluid-model formulas, algorithms, and theorem statements.
- Reorganized the appendix proofs so that the main reductions, memory terms, and stochastic bounds are easier to verify.

Closing

We again thank the Area Editor, the Associate Editor, and the referees for their careful and constructive guidance. Their comments led us to make a substantial revision rather than a local edit: the manuscript now frames the problem around exogenous arrivals, endogenous KV-cache memory growth, eviction-induced restarts, and latency; states the modeling scope and memory mechanism more explicitly; presents WAIT and Nested WAIT as fluid-guided scheduling policies that reduce the memory-coupled state space to threshold and boundary queues; and evaluates the policies on both synthetic and real-data workloads, using clearer visualizations and adding long-horizon checks and supplemental real-GPU validation. We hope the revised manuscript now conveys both the contribution and the scope of the results more clearly.

References

- Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.
- Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*.
- Atharva (2026) The kv cache: How it eliminates redundancy. <https://huggingface.co/blog/atharv6f/kv-cache-basics>, accessed: 2026-05-01.
- Hugging Face (2025) Llama 2 model documentation. https://huggingface.co/docs/transformers/model_doc/llama2, accessed: 2026-05-01.
- NVIDIA (2026) Nvidia a100 tensor core gpu specifications. <https://www.nvidia.com/en-eu/data-center/a100/>, accessed: 2026-05-01.
- Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.
- SGLang Team (2024) Sglang: Fast serving framework for large language models and vision language models. <https://github.com/sgl-project/sglang>, gitHub repository.
- Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.